

AgriAgent

FYP Research Bible

An Autonomous Multi-Agent Vision Framework for
Zero-Shot Weed-Crop Segmentation and Site-Specific Spray Mapping

Instructor	Prof. V. D. Dhore
Domain	Image Segmentation + Object Detection in Agriculture
Framework	SmartDesk (LangGraph) + Grounding DINO + SAM 2
Team Size	4 members
Academic Year	2025-26
Deliverable	Published paper + working system + thesis

This document serves as the team's internal research bible and planning reference. It covers all models, datasets, papers, feasibility, publishability, and implementation roadmap required to execute the FYP.

1. Project Overview & Problem Statement

Conventional weed management in agriculture relies on two approaches: manual scouting by trained agronomists, which is labor-intensive and subjective, and blanket herbicide spraying across entire fields, which causes significant chemical overuse, crop damage from off-target application, environmental contamination of soil and waterways, and the accelerating evolution of herbicide-resistant weed biotypes. According to Iowa State University research, precision (site-specific) spraying can reduce herbicide usage by an average of 76%, with savings reaching 90% under low weed pressure conditions, while maintaining equivalent weed control efficacy compared to broadcast application. The economic and environmental case for targeted spraying is overwhelming, yet adoption remains limited by the cost and inflexibility of current computer vision solutions.

Existing CNN-based segmentation models for crop-weed discrimination, such as U-Net, DeepLabv3+, and their variants, require costly pixel-level annotation for every new crop-weed combination and extensive retraining when deployed to new geographic regions, crop types, or growth stages. A model trained on sugar beet fields in Germany will fail on cotton fields in India without complete retraining. This domain specificity creates a deployment bottleneck that prevents scalable adoption of precision weed management technology.

1.1 Thesis Title

AgriAgent: An Autonomous Multi-Agent Vision Framework for Zero-Shot Weed-Crop Segmentation and Site-Specific Spray Mapping

1.2 Formal Problem Statement

This project proposes a multi-agent orchestration framework built on LangGraph (extending the existing SmartDesk codebase) in which a natural-language command such as "segment all weeds between the crop rows and generate a spray map" autonomously triggers three cascaded vision agents: (1) an open-vocabulary grounding agent using Grounding DINO-T that localizes objects by free-text description, (2) a promptable segmentation agent using SAM 2 that converts bounding boxes into pixel-accurate masks, and (3) an analysis agent that computes per-instance weed/crop masks, derives a site-specific spray map with GPS coordinates, and estimates herbicide savings percentage. The critical innovation is that this entire pipeline operates **without any task-specific retraining** on the target agricultural dataset, relying instead on the zero-shot transfer capabilities of Grounding DINO and the universal segmentation capability of SAM 2.

1.3 Five Application Domains

The framework supports five distinct agricultural applications, all sharing the same underlying pipeline but activated by different natural-language prompts. This "same pipeline, different prompt" design demonstrates zero-shot generalization, which constitutes the primary academic novelty of this work.

#	Application	Example Prompt	Output Artifact	Dataset
1	Site-Specific Weed Spraying	"Find all weeds and spare the crops"	Weed vs. crop pixel masks + spray map	SugarBeets, DeepWeeds
2	Disease & Pest Lesion Auditing	"Highlight diseased leaf regions"	Lesion area % per leaf + severity report	PlantDoc, PlantVillage
3	Yield Estimation / Fruit Counting	"Count the wheat heads"	Count + size distribution	GWHD 2021
4	Canopy Coverage & Stand Count	"Measure crop coverage vs. bare soil"	Germination health % + gap alerts	SugarBeets, drone imagery
5	Nutrient Stress (Chlorosis) Mapping	"Segment yellowing/stressed leaves"	Stress heatmap + recommendation	PlantDoc, self-labeled

Table 1: The five application domains. All share the same Grounding DINO + SAM 2 + Analysis pipeline.

1.4 Why Precision Agriculture?

Precision agriculture is the ideal domain for this FYP for several converging reasons. First, weeds and crops are visually distinct in field imagery: green vegetation against brown soil creates strong contrast that SAM 2 segments reliably without any fine-tuning. Second, all required datasets are publicly available with permissive licenses, eliminating the data acquisition bottleneck. Third, no medical or PhD-level domain expertise is needed to interpret results; a weed mask is visually self-explanatory. Fourth, the real-world impact story "reducing herbicide use by 76-90%" is compelling and directly provides the thesis conclusion. Fifth, the instructor explicitly advised an agriculture-based problem, aligning the project with domain expectations.

2. Foundation Models Deep Dive

This section provides exhaustive technical specifications for the three foundation models that form the backbone of the AgriAgent pipeline. Understanding their architectures, parameter counts, inference speeds, and zero-shot capabilities is essential for feasibility assessment and implementation planning.

2.1 Grounding DINO (Open-Vocabulary Object Detector)

Paper: "Marrying DINO with Grounded Pre-Training for Open-Set Object Detection" by Shilong Liu et al., ECCV 2024. Citations: 5,920. Grounding DINO unifies the DINO detector architecture with grounded pre-training on large-scale image-text pairs (Objects365 + GoldG datasets), enabling open-vocabulary object detection where any free-form text description (e.g., "weed", "yellow leaf", "diseased spot") can be used as a detection query without any fine-tuning on the target domain.

Architecture: The model consists of four key modules: (1) a text backbone (BERT or RoBERTa) that encodes the input text prompt into token-level and sentence-level features, (2) an image backbone (Swin Transformer) that extracts multi-scale visual features, (3) a Feature Enhancer that performs bidirectional cross-attention between language and vision features at multiple scales, and (4) a Language-Guided Query Selection module that initializes detection queries by aligning text and image features, followed by a deformable transformer decoder that refines these queries into final bounding boxes with associated text phrases.

Variant	Parameters	COCO Zero-Shot	ODinW Zero-Shot	Speed
Grounding DINO-T (Swin-T)	~172M	52.5 AP (COCO zs)	26.1 AP (ODinW zs)	~74 FPS (V100)
Grounding DINO-B (Swin-B)	~227M	56.4 AP (COCO zs)	28.4 AP (ODinW zs)	~30 FPS (V100)
Grounding DINO 1.6 Edge	varies	55.4 AP (COCO)	36.1% (Roboflow)	>10 FPS (edge)

Table 2: Grounding DINO model variants and benchmarks.

ODinW (Object Detection in the Wild): This benchmark, introduced by Xu et al. (NeurIPS 2023), evaluates zero-shot object detection across 35 real-world datasets including agricultural domains. Grounding DINO achieves 26.1 AP mean across all 35 datasets in zero-shot mode. Critically, its performance on the PlantDoc agricultural dataset within ODinW is relatively low compared to COCO, which directly motivates our refinement agent. The ODinW paper (cited by 80) provides the formal evaluation framework we will use for benchmarking.

Colab T4 Compatibility: Grounding DINO-T (172M params) fits comfortably within the 16GB VRAM of a free Google Colab T4 GPU. With half-precision (FP16) inference, memory usage is approximately 350MB for the model weights plus dynamic memory for the attention maps. Inference on a 1024x1024 image takes approximately 0.8 seconds on T4, which is sufficient for interactive demonstration and batch processing of field images.

2.2 SAM 2 (Segment Anything Model 2)

Paper: "SAM 2: Segment Anything in Images and Videos" by Nitin Ravi et al., Meta AI, 2024. Citations: 5,733. SAM 2 is a foundation model for promptable visual segmentation that operates on both images and video, handling object occlusion and reappearance across frames. It is 6 times faster than the original SAM while being more accurate, making it practical for real-time agricultural applications.

Architecture: SAM 2 uses a streaming memory attention mechanism that stores embeddings of previously segmented objects and attends to them when processing new frames. For single-image segmentation (our primary use case), it accepts three types of prompts: (1) point prompts (click on object/background), (2) box prompts (bounding box from Grounding DINO), and (3) mask prompts (coarse mask from a previous step). The model outputs pixel-level masks with a predicted Intersection-over-Union (IoU) confidence score, which is the key signal used by our refinement agent to decide whether the segmentation is satisfactory or needs corrective prompting.

Variant	Parameters	Speed (A100)	Speed (T4)	Notes
SAM 2 Tiny	38.9M	47.2 FPS (A100)	~8 FPS (T4)	Smallest; sufficient for most agri tasks
SAM 2 Small	~46M	~40 FPS (A100)	~5 FPS (T4)	Better accuracy on small objects
SAM 2 Base+	~88M	30-40 FPS (A100)	~3 FPS (T4)	Best quality/speed tradeoff
SAM 2 Large	~224M	~20 FPS (A100)	~1 FPS (T4)	Maximum accuracy; may OOM on T4

Table 3: SAM 2 model variants. Tiny is recommended for Colab T4 deployment.

Recommended Configuration: SAM 2 Tiny (38.9M params) is the recommended variant for the FYP. It runs at approximately 8 FPS on a Colab T4 GPU, processes 1024x1024 images in under 125ms, and provides segmentation quality that is more than sufficient for crop-weed discrimination where the visual contrast between green vegetation and brown soil is inherently strong. The predicted IoU score from SAM 2 is used as the quality gate in the refinement loop: if predicted IoU is below 0.85, the analysis agent identifies the centroid of the mask error region and adds a corrective point prompt, iterating up to 2 times.

2.3 CLIPSeg (Zero-Shot Image Segmentation)

Paper: "Image Segmentation Using Text and Image Prompts" by Tim Lueddecke and Alexander Ecker, CVPR 2022. Citations: 1,030. CLIPSeg extends the CLIP vision-language model with a Transformer-based decoder for dense pixel-level prediction. Given a text prompt like "weed" or "diseased leaf", it produces a coarse segmentation mask without any fine-tuning. However, its masks are significantly less precise than SAM 2, making it unsuitable as the primary segmentation agent. It is included as a potential fallback and as a point of comparison for the novelty claim: AgriAgent achieves better segmentation quality than CLIPSeg by chaining Grounding DINO (precise bounding boxes) with SAM 2 (precise masks) rather than using CLIPSeg's coarse decoder directly.

3. Agricultural Datasets Catalog

All datasets listed below are publicly available with permissive licenses, require no labeling work for evaluation (ground truth masks or bounding boxes are provided), and can be downloaded without registration fees. This eliminates the data acquisition bottleneck entirely.

3.1 Sugar Beets 2016 (University of Bonn)

Captured by an unmanned ground vehicle (UGV) in sugar beet fields near Bonn, Germany, as part of the PhenoRob research group. Contains 25,429 field images with pixel-wise semantic annotations for two classes: sugar beet crop and weed. WeedNet-R achieves 85.70% AP for weed and 98.89% AP for sugar beet (Frontiers in Plant Science, 2023, cited by 36). Milioto et al. (cited by 219) demonstrates real-time blob-wise classification. Ideal for Application 1 (weed spraying) and Application 4 (canopy coverage). **Download:** DatasetNinja (sugar-beets-2016), PhenoRoam portal.

3.2 DeepWeeds

Published by Alex Olsen et al. in 2019 (cited by 659). Contains 17,509 images of 256x256 pixels across 9 classes: 8 nationally significant weed species from northern Australia plus a negative class. Train/test split: 15,007/2,501. Primary challenge: inter-class similarity and intra-class dissimilarity. ADeepWeeD (2025, cited by 13) provides an adaptive baseline. Suitable for Application 1 with species-level classification. **Download:** GitHub (AlexOlsen/DeepWeeds), Kaggle (imsparsh/deepweeds), TensorFlow Datasets.

3.3 PlantDoc

Contains 2,598 images across 13 plant species and 27-29 disease categories, with 2,482 images having bounding boxes (8,595 total labeled objects). Annotation: ~300 human hours. Primary dataset for Application 2 (disease auditing). Included in ODinW benchmark where Grounding DINO shows low zero-shot performance, motivating our refinement agent. YOLO-LeafNet (2025) provides supervised baseline. **Download:** DatasetNinja (plantdoc), Roboflow (improved-plantdoc).

3.4 Global Wheat Head Detection (GWHD 2021)

Published by David et al. in 2021 (cited by 184). Contains 6,515 PNG images with over 300,000 labeled wheat heads as bounding boxes, sourced from 12 countries. Kaggle competition baseline: 0.677 (YOLOv11). Primary benchmark for Application 3 (yield estimation). Multi-country diversity tests zero-shot generalization. **Download:** Kaggle (vbookshelf/global-wheat-head-dataset-2021).

3.5 PlantVillage

Contains 54,303 single-leaf images against uniform backgrounds across 14 crop species and 38 classes (26 diseases + 12 healthy). Documented bias issues: lab-style backgrounds differ from field conditions. Useful for Application 2 as pre-training and Application 5 (nutrient stress). Graph-based DL model (2024, cited by 63) provides supervised baseline. **Download:** GitHub (spmohanty/plantvillage-dataset), Kaggle (emmarex/plantdisease).

3.6 Roboflow Universe (Supplementary)

Community-contributed agricultural datasets with pre-trained models: soy-weed-seg (1,000 images), weed detection in agriculture (1,300 images), peanut-and-weed (instance segmentation). Supplements primary benchmarks for cross-dataset validation and demo construction. **Access:** universe.roboflow.com/browse/agriculture.

Dataset	Images	Classes	Annotations	Target Apps	License
SugarBeets 2016	25,429	2 (crop, weed)	Pixel masks	App 1, 4	Free (PhenoRob)
DeepWeeds	17,509	9 (8 weeds + neg)	Image-level	App 1	Free (Olsen 2019)
PlantDoc	2,598	27-29 (diseases)	Bounding boxes	App 2, 5	Free (Roboflow)
GWHD 2021	6,515	1 (wheat head)	Bounding boxes	App 3	Free (Kaggle)
PlantVillage	54,303	38 (species_disease)	Image-level	App 2, 5	Free (GitHub)
Roboflow Universe	varies	varies	Masks/bboxes	All (suppl.)	Free (Roboflow)

Table 4: Complete dataset catalog.

4. Five Application Deep Dives

Each application follows the same AgriAgent pipeline: user prompt + image is processed by the Orchestrator, which dispatches to the Grounding Agent, Segmentation Agent, Analysis Agent, and Report Agent. The difference between

applications lies entirely in the text prompt and the post-processing logic, demonstrating the zero-shot generalization thesis.

4.1 Site-Specific Weed Spraying

Prompt: "Find all weeds and spare the crops" | **Segments:** Weeds vs. crop plants | **Output:** Spray map (GPS coordinates + per-patch classification) + estimated herbicide savings %

The Grounding Agent receives "weed. crop." and produces bounding boxes. SAM 2 converts each box into a pixel mask. The Analysis Agent computes herbicide savings: $1 - (\text{weed_mask_area} + 5\text{cm_buffer}) / \text{total_field_area}$. The Report Agent generates a spray map overlay (green=crop, red=weed) and CSV with per-patch coordinates. On SugarBeets 2016, supervised WeedNet-R achieves 85.70% weed AP; our zero-shot approach targets even 70% of that quality, yielding estimated 53-63% herbicide savings, which is compelling for a zero-shot system.

4.2 Disease & Pest Lesion Auditing

Prompt: "Highlight diseased leaf regions" | **Segments:** Lesions/spots on leaves | **Output:** Severity report (lesion area % per leaf) with disease classification

The Grounding Agent receives "diseased leaf. spot. lesion. yellow patch." The Analysis Agent computes lesion area ratio per leaf, producing severity scores (0-100%). Leaves above 30% severity are flagged for treatment. PlantDoc provides the primary evaluation dataset with bounding boxes for 27 disease categories across 13 species. The supervised YOLO-LeafNet baseline provides the comparison point for zero-shot evaluation.

4.3 Yield Estimation / Fruit Counting

Prompt: "Count the wheat heads" | **Segments:** Individual fruits/heads | **Output:** Count + size distribution histogram + yield estimate

Grounding DINO produces per-instance bounding boxes; the count equals the number of boxes. SAM 2 refines each into a mask for size distribution. Yield estimate = count x average weight per fruit (configurable). GWHD 2021 is the primary benchmark (300k+ wheat heads, 12 countries). Metric: MAE vs. ground truth count. The supervised Kaggle baseline is 0.677 mAP (YOLOv11); achieving 50-60% counting accuracy zero-shot would be strong.

4.4 Canopy Coverage & Stand Count

Prompt: "Measure crop coverage vs. bare soil" | **Segments:** Canopy vs. soil gaps | **Output:** Germination health % + gap location alerts

The Grounding Agent receives "crop. bare soil. gap." The Analysis Agent computes canopy coverage ratio and identifies gaps (contiguous soil regions > 50cm diameter) indicating failed germination. Output: germination health score (0-100%) + gap GPS coordinates for re-seeding. SugarBeets and drone imagery serve as evaluation data. This application is particularly valuable for early-season monitoring when re-seeding decisions are time-critical.

4.5 Nutrient Stress (Chlorosis) Mapping

Prompt: "Segment yellowing/stressed leaves" | **Segments:** Yellow vs. healthy foliage | **Output:** Stress heatmap + fertilizer recommendation

Chlorosis indicates nutrient deficiency (nitrogen, iron, or magnesium). The Grounding Agent receives "yellow leaf. stressed leaf. chlorosis." The Analysis Agent creates a spatial heatmap and optionally queries the Knowledge/RAG Agent for fertilizer recommendations from an agronomy handbook. This demonstrates integration of the vision pipeline with SmartDesk's existing RAG capabilities, combining visual analysis with domain knowledge retrieval for actionable recommendations. PlantDoc and self-labeled images provide evaluation data.

5. Existing Literature & Gap Analysis

This section surveys the current state of agricultural computer vision and identifies the specific gaps that AgriAgent addresses. This gap analysis is the heart of the FYP: the thesis must demonstrate that existing approaches have limitations, and that AgriAgent provides a novel and meaningful improvement.

5.1 Supervised CNN-Based Segmentation

The dominant paradigm in agricultural segmentation is supervised encoder-decoder training on pixel-annotated field images. U-Net++ achieves 96.12% accuracy on crop-weed data (Fathipoor et al., 2023, cited by 34). DeepLabv3+ achieves 85.57 mIoU on corn-weed segmentation (DSC-DeepLabv3+, cited by 12). Swin-UNet outperforms DeepLabv3+ and Mask R-CNN on corn weed datasets (cited by 35). These models achieve excellent performance but require complete retraining for every new crop-weed combination, geographic region, or growth stage, which is the fundamental scalability limitation that AgriAgent addresses.

5.2 Open-Vocabulary Detection

Grounding DINO (ECCV 2024, cited by 5,920) and YOLO-World (CVPR 2024) represent the open-vocabulary detection paradigm. Both achieve strong zero-shot performance on COCO (Grounding DINO: 52.5 AP, YOLO-World: 35.4 mAP) but significantly lower on ODinW (26.1 AP for Grounding DINO). This performance gap on agricultural datasets motivates the need for a refinement mechanism. No existing work combines open-vocabulary detection with an autonomous refinement loop for agricultural applications.

5.3 Zero-Shot Segmentation

CLIPSeg (CVPR 2022, cited by 1,030) provides zero-shot segmentation via text prompts but produces coarse masks. Belissent et al. (2024, cited by 55) apply transfer and zero-shot learning to weed detection. Chong et al. (2025, University of Bonn) propose zero-shot anomaly segmentation for crop-weed discrimination without training, which is the closest existing work. However, their method uses a fixed anomaly detection formulation rather than the flexible natural-language prompting and multi-agent refinement loop that AgriAgent provides.

5.4 Agent-Based Segmentation

MedSAM-Agent (February 2026) reformulates interactive segmentation as multi-step autonomous decision-making with reinforcement learning. Its key innovation is the multi-turn targeted refinement loop. AgriAgent adapts this concept but replaces the RL-trained correction policy with a simple heuristic based on SAM 2's predicted IoU score (if $\text{IoU} < 0.85$, add a corrective point at the mask error centroid), requiring no RL training. This heuristic is inspired by MedSAM-Agent but implemented in approximately 30 lines of NumPy code.

5.5 Identified Gaps (The FYP Core)

Gap	Description	Evidence	AgriAgent Solution
G1	No zero-shot agri framework	All methods require supervised training on target domain	AgriAgent uses Grounding DINO + SAM 2 with zero-shot transfer
G2	No multi-agent orchestration	No work uses agent-based frameworks for agri CV	LangGraph orchestrates specialized agents with NL dispatch
G3	No refinement without RL	MedSAM-Agent uses RL; too complex for UG project	IoU-based heuristic refinement (30 lines NumPy)
G4	No spray map generation	Segmentation papers stop at mask output	Analysis Agent computes spray coordinates + savings estimate

Gap	Description	Evidence	AgriAgent Solution
G5	No cross-app generalization	Each paper addresses one application only	Same pipeline, 5 applications, different prompts

Table 5: The five identified gaps that AgriAgent addresses.

6. Multi-Agent Architecture (AgriAgent)

AgriAgent extends the SmartDesk LangGraph framework by adding vision-specific agents and state fields to the existing Orchestrator-Worker pattern. SmartDesk uses a hierarchical supervisor-worker architecture where the Orchestrator assigns one task at a time to specialized agents (WorkspaceAgent, KnowledgeAgent, ProductivityAgent), each compiled as a LangGraph subgraph with its own message channel for isolation. The vision agents follow this same pattern.

6.1 Pipeline Flow

For a weed segmentation query ("Segment weeds and generate spray map" + field image), the flow is: (1) The user submits via the Streamlit UI. (2) The Orchestrator parses the command and determines the agent sequence: Grounding Agent, Segmentation Agent, Analysis Agent, Report Agent. (3) The Grounding Agent receives the image and text prompt "weed. crop." via Grounding DINO-T, producing bounding boxes with labels and confidence scores. (4) The Segmentation Agent invokes SAM 2-Tiny with box prompting, producing pixel masks and predicted IoU. (5) The Analysis Agent computes weed/crop instances, mask areas, herbicide savings, and checks IoU. If any mask has predicted IoU below 0.85, it identifies the mask error centroid and adds a corrective point prompt, re-triggering the Segmentation Agent (max 2 iterations). (6) The Report Agent generates the spray map overlay and CSV/PDF report.

6.2 State Schema Extensions

The existing SmartDesk GraphState (TypedDict in state.py) is extended with these fields:

Field	Type	Purpose
image	Optional[PIL.Image]	Input field image from user upload
boxes	List[Box]	Bounding boxes from Grounding DINO (x1,y1,x2,y2,label,score)
masks	List[Mask]	Pixel masks from SAM 2 (binary array + predicted IoU)
refinement_count	int	Number of refinement iterations (max 2)
spray_map	Optional[Dict]	GPS-tagged spray coordinates + savings %
analysis_report	Optional[Dict]	Per-app metrics (counts, areas, severity)

Table 6: GraphState extensions for the vision pipeline.

6.3 Agent Definitions

Agent	Model	Input	Output	Role
Grounding Agent	Grounding DINO-T	image + text prompt	List[Box]	Open-vocab object detection
Segmentation Agent	SAM 2-Tiny	image + Box/Point	List[Mask]	Promptable instance segmentation
Analysis Agent	OpenCV + NumPy	List[Mask] + metadata	spray_map + report	Geometry, savings, refinement trigger
Report Agent	SmartDesk Workspace	analysis_report + masks	Overlay + CSV/PDF	Visual overlay + report generation

Table 7: Agent definitions and roles.

6.4 Refinement Loop Detail

When SAM 2 produces a mask with predicted IoU below 0.85, the Analysis Agent computes the error centroid: the point on the mask boundary farthest from the nearest Grounding DINO box edge. This point is classified as positive (inside) or negative (outside) based on whether the mask under-segments or over-segments. The Segmentation Agent is re-invoked with the original box plus the corrective point. This is inspired by MedSAM-Agent's multi-turn refinement but uses a deterministic heuristic (IoU threshold + error centroid) instead of a learned RL policy, reducing implementation from thousands of lines of RL training code to approximately 30 lines of NumPy. Maximum 2 iterations prevents infinite loops while allowing sufficient correction.

6.5 SmartDesk Codebase Architecture (Current State)

SmartDesk is a hierarchical multi-agent AI assistant orchestrated via LangGraph, powered by the Groq API (GPT-OSS-120B model). It follows a **Supervisor-Worker (Orchestrator-SubAgent)** pattern: the Orchestrator analyzes user queries, generates a step-by-step plan as Task objects, routes execution to specialized workers, and aggregates their Artifacts into structured responses. Each sub-agent is a localized ReAct (Reasoning + Acting) loop with a specific domain of tools, compiled as a LangGraph subgraph with its own message channel for isolation.

Existing Sub-Agents

Agent	Domain	Capabilities	Engineering Safeguards
WorkspaceAgent	Local file I/O	List dirs, read/write files, search directory	Token limit patching (ignores .venv, __pycache__), multi-file RAW_CONTEXT artifacts
KnowledgeAgent	RAG / Vector Search	Ingest PDFs/Markdown/TXT from KnowledgeBase/ folder	ChromaDB with metadata-diffing; only re-indexes new/modified files
ProductivityAgent	External APIs	Gmail, Google Calendar/Docs/Sheets, Telegram	Mock Mode fallback when credentials missing; email hard-cap (10 max)

Table 7a: Existing SmartDesk sub-agents that AgriAgent extends.

Global State (state.py)

The **GraphState** is a TypedDict at the core of the LangGraph implementation. Every node reads from and writes to this state. Key existing fields include: **current_task** and **completed_tasks** (the Orchestrator's task queue), **messages** (conversation history between user and Orchestrator), **workspace_messages**, **knowledge_messages**, **productivity_messages** (isolated message histories per sub-agent to prevent cross-contamination), **artifacts** (structured dictionary where agents drop findings instead of polluting chat history with massive text dumps), and **checkpointner** (MemorySaver instance that persists graph state across turns based on a thread_id, e.g., Streamlit chat_id). The vision pipeline extensions (Table 6) are added to this existing TypedDict.

Clean Slate Finalizers

SmartDesk solves context contamination with **Clean Slate Finalizers**: at the end of every agent's task, a finalizer node runs that uses LangChain's **RemoveMessage** to delete all intermediate scratchpad thoughts and tool calls from that specific task run. When the agent is called again for a new task, it starts with a completely clean slate, armed only with the new instructions. This prevents agents from confusing current instructions with stale messages from previous tasks. For AgriAgent, the vision agents (Grounding, Segmentation, Analysis, Report) will each have their own finalizer, ensuring that per-image processing state does not leak between successive field images in batch mode.

Artifact System

Instead of dumping raw text into chat history (which destroys the context window), agents yield **Artifacts** such as RAW_CONTEXT, STATUS, SEARCH_RESULTS. These are structured objects stored in GraphState. The Orchestrator

references artifacts by their ID rather than reading entire raw text. For AgriAgent, the vision pipeline adds artifact types: VISION_MASKS (binary mask arrays), BOUNDING_BOXES (detection results), SPRAY_MAP (GPS-tagged coordinates), and ANALYSIS_REPORT (per-app metrics). This keeps the Groq LLM context window pristine even when processing high-resolution field images that produce large mask arrays.

Mock Mode Fallbacks

SmartDesk's ProductivityAgent features a resilient fallback mechanism: if Google API credentials are missing or have mismatched scopes, it gracefully falls back to Mock Mode using local JSON files without crashing. AgriAgent adopts this same pattern for the vision pipeline: if GPU inference is unavailable (e.g., deployment on CPU-only server), the Grounding Agent falls back to CLIPSeg for coarse zero-shot segmentation, and the Segmentation Agent skips refinement. This ensures the Streamlit demo never hard-crashes, even without a GPU.

7. Feasibility, Costs & Computational Requirements

7.1 GPU Requirements

Component	Parameters	GPU Memory	Latency	Hardware
Grounding DINO-T inference	172M params	~350MB FP16	~0.8s/image	Colab T4 (16GB)
SAM 2-Tiny inference	38.9M params	~80MB FP16	~125ms/mask	Colab T4 (16GB)
Combined pipeline (1 image)	~211M total	~430MB FP16	~1.2s/image	Colab T4 (16GB)
Refinement loop (2 iters)	Same models	Same memory	+0.5s/refinement	Colab T4 (16GB)

Table 8: GPU requirements. Total footprint well within Colab T4 limits.

7.2 Cost Analysis

Resource	Tier/Limit	Cost	Purpose
Google Colab (T4 GPU)	Free: 12hr/day	\$0	Primary development + demo
Groq API (LLM)	Free: 30 req/min	\$0	Agent orchestration + RAG
HuggingFace Inference	Free: rate-limited	\$0	Embeddings for RAG
Dataset downloads	All public	\$0	All 5+ datasets
ChromaDB (vector store)	Local SQLite	\$0	Document indexing for RAG
Streamlit hosting	Community Cloud	\$0	Demo deployment
Total estimated cost		\$0	Entire project at zero cost

Table 9: Complete cost analysis. The entire FYP runs at zero monetary cost.

7.3 Doability Assessment

The project is highly doable for a team of 4 within a 16-week semester. Grounding DINO-T + SAM 2-Tiny (211M total params) run inference on a free Colab T4 in approximately 1.2 seconds per image. No custom training is required. All datasets are publicly available with ground truth. Prompts are simple nouns. SmartDesk provides the LangGraph orchestration, RAG, and workspace infrastructure, reducing scope to four new vision agents. The live demo is bulletproof: plants on soil segment cleanly with SAM 2 virtually every time.

Challenges: Grounding DINO may produce spurious detections on unfamiliar imagery; the refinement loop and confidence thresholding mitigate this. SAM 2 may merge overlapping weeds into one mask; per-box prompting (each box gets its own SAM 2 call) handles this. The refinement loop may not show statistically significant improvement; the

multi-application generalization serves as the backup novelty claim.

8. Publishability Strategy

8.1 Novelty Claim

Primary novelty: AgriAgent is the first multi-agent framework combining open-vocabulary detection (Grounding DINO) with promptable segmentation (SAM 2) and an autonomous IoU-based refinement loop for zero-shot agricultural image analysis, producing actionable outputs (spray maps, severity reports, yield estimates) without any task-specific training.

Secondary novelty: Five distinct agricultural applications handled by the same pipeline with only prompt changes, establishing zero-shot generalization across tasks rather than just across datasets.

8.2 Comparison to Closest Prior Work

Method	Models	Paradigm	Apps	Actionable Output
AgriAgent (ours)	Grounding DINO + SAM 2	Zero-shot + refinement	5 applications	Yes (spray map)
Chong et al. 2025	Anomaly detection	Zero-shot (no refinement)	1 (crop-weed)	No
Belissent et al. 2024	Transfer learning	Few-shot fine-tuned	1 (weed detection)	No
MedSAM-Agent 2026	SAM 3 + RL	RL-based refinement	1 (medical)	No
YOLO-World 2024	YOLO + CLIP	Zero-shot detection only	1 (detection)	No
U-Net++ baseline	Supervised U-Net++	Full supervision	1 (crop-weed)	No

Table 10: Comparison to closest prior work.

8.3 Target Venues

Primary: CVPR Agriculture-Vision Workshop (7th edition, CVPR 2026). Focused on CV in agriculture, IEEE-indexed proceedings, estimated 40-50% acceptance rate. The 6th edition was held at CVPR 2025, confirming it is active and recurring. **Secondary:** MDPI AgriEngineering Special Issue on "Applications of Computer Vision in Agriculture." Open access, reasonable fees (~1000-2000 CHF, potentially waived for students). **Tertiary:** ECVA or similar regional venue as lower-barrier fallback.

8.4 What Makes This Publishable

The work is publishable rather than incremental for three reasons: (1) the combination of open-vocabulary detection with promptable segmentation and autonomous refinement has not been applied to agriculture, making this a genuine architectural contribution; (2) demonstrating zero-shot generalization across five applications provides broader empirical evidence than typical single-task papers; (3) producing actionable outputs (spray maps with herbicide savings) goes beyond segmentation-only outputs and demonstrates practical impact valued by the Agriculture-Vision community. The main risk is that the refinement loop may not produce statistically significant improvements; this must be demonstrated with confidence intervals.

9. Implementation Roadmap & Risk Assessment

9.1 Team Member Roles

Member	Role	Responsibilities
Member 1	Architect / Orchestrator	Extend GraphState, add vision agents to LangGraph, implement Orchestrator routing, project infrastructure
Member 2	Vision Pipeline Engineer	Grounding DINO-T inference, SAM 2-Tiny inference, box-to-mask, refinement loop, confidence thresholding
Member 3	Data & Evaluation Lead	Dataset download/preprocessing, IoU/Dice scripts, benchmarks vs baselines, statistical testing, thesis figures
Member 4	UI / Demo Engineer	Streamlit interface (upload, prompt, overlay, download), Report Agent (CSV/PDF), live demo preparation

Table 11: Team member roles.

9.2 Week-by-Week Roadmap

Weeks	Phase	Tasks	Lead
1-2	Setup & Exploration	Clone SmartDesk, Colab setup, Grounding DINO + SAM 2 on sample images, dataset downloads, Git repo	All (parallel)
3-4	Core Pipeline	Grounding Agent + Segmentation Agent as LangGraph nodes, GraphState extensions, basic pipeline	Members 1+2
5-6	Analysis & Refinement	Analysis Agent (IoU check, mask area, savings), refinement loop, SugarBeets evaluation	Members 2+3
7-8	Applications 2-5	Adapt pipeline for disease, yield, canopy, stress with different prompts	Members 1+2
9-10	Evaluation & Baselines	Full evaluation on all datasets, supervised baselines, statistical significance, thesis figures	Member 3
11-12	UI & Demo	Streamlit interface, Report Agent, spray map overlay, demo testing	Member 4
13-14	Integration & Testing	E2E integration, bug fixes, speed optimization, code documentation	All
15-16	Thesis & Submission	Thesis chapters, Agriculture-Vision paper, presentation slides, demo rehearsal	All

Table 12: Week-by-week roadmap for 16-week semester.

9.3 Risk Assessment

Risk	Severity	Mitigation	Contingency
GPU OOM on T4	Medium	FP16 inference; CPU fallback for SAM 2; reduce resolution	Monitor memory; have CPU path by Week 3
Grounding DINO hallucination	High	Filter by confidence (0.3); validate against expected count range	Conservative threshold; NMS with low IoU (0.3)
SAM 2 over-segmentation	Medium	Use per-box prompting instead of multi-mask mode	Each box gets independent SAM 2 call
Dataset bias (PlantVillage)	Low	Use PlantDoc as primary for App 2	Document bias; evaluate on both
API rate limits (Groq/HF)	Low	Cache LLM responses; use local LLM fallback (--local)	SmartDesk already has local fallback
Integration failure	Medium	Test incrementally with mock agents	Week 3-4 integration sprint

Risk	Severity	Mitigation	Contingency
Insufficient novelty	Medium	Ablation: with vs without refinement	Pivot to multi-app generalization as primary novelty

Table 13: Risk assessment with mitigation and contingency plans.

10. Thesis Metrics & Evaluation Plan

10.1 Segmentation Metrics

Dice Coefficient: $\text{Dice} = 2|P \cap G| / (|P| + |G|)$. We report per-class Dice (weed, crop) and mean Dice. **IoU:** $\text{IoU} = |P \cap G| / |P \cup G|$. We report per-class IoU and mIoU as the primary segmentation quality metric, consistent with agricultural segmentation literature.

10.2 Detection Metrics

Precision / Recall / AP: Computed at IoU threshold 0.5 following COCO protocol, on PlantDoc and GWHD where bounding box ground truth is available.

10.3 Application-Specific Metrics

Application	Metric	Target	Dataset
App 1: Weed Spraying	$\text{Savings \%} = 1 - (\text{weed_area} + \text{buf}) / \text{total}$	>60% savings, <10% crop damage	SugarBeets + DeepWeeds
App 2: Disease Auditing	$\text{Lesion \%} = \text{lesion_mask} / \text{leaf_mask}$	Within 10% of manual assessment	PlantDoc + PlantVillage
App 3: Yield Counting	$\text{MAE} = \text{mean}(\text{pred} - \text{true})$	$\text{MAE} < 5$ per image	GWHD 2021
App 4: Canopy Coverage	$\text{Coverage \%} = \text{canopy} / \text{total pixels}$	Within 5% of manual grid	SugarBeets + drone
App 5: Stress Mapping	$\text{Stress \%} = \text{stressed} / \text{foliage}$	Correlated with manual rating ($r > 0.8$)	PlantDoc + self-labeled

Table 14: Application-specific metrics and targets.

10.4 Statistical Significance

All metrics are computed over full test sets with mean and 95% confidence intervals (bootstrapped, 1000 resamples). For refinement ablation: paired t-test on per-image Dice, reporting p-value and Cohen's d. Significance threshold: $p < 0.05$. For supervised comparisons: Wilcoxon signed-rank test (non-parametric). All evaluation code in standalone iou_dice.py for reproducibility.

11. Alternative FYP Ideas & Strategic Options

If the primary "Smart Farm Commander" (5-app AgriAgent) proves too broad for the available timeline, the following five highly focused alternatives use the **exact same technology stack** (Grounding DINO + SAM 2 + LangGraph) but solve different, equally publishable problems. Each alternative narrows scope to one application while deepening the contribution, which may actually improve publishability at top venues that value depth over breadth. The team should discuss these with Prof. Dhore in Week 1 before committing.

11.1 Alternative 1: Drone Scout (Macro-Level Weed Mapping)

Problem: Farmers fly drones over fields but spend hours manually reviewing orthomosaic (stitched drone) images to locate weed patches. Manual annotation of drone imagery at field scale is impractical due to the sheer area covered (hundreds of acres per flight). **Vision AI:** Semantic segmentation of orthomosaic images into "Green Crop Rows" vs "Brown Soil" vs "Random Weed Patches." Grounding DINO prompts: "weed patch. crop row. bare soil." **Agent Action:** The agent calculates GPS coordinates of weed patches from image geotags and generates a flight path for a spray drone to visit only those coordinates. **Dataset:** SugarBeets 2016 (has vehicle/drone imagery) or WeedMap dataset. **Publishability:** Moving from micro (leaf-level) to macro (field-level) segmentation is a massive trend in precision agriculture. Demonstrating zero-shot segmentation on orthomosaic images (which have very different statistics than close-up photos) tests the true generalization capability of the pipeline.

11.2 Alternative 2: Smart Harvester (Fruit Ripeness Grading)

Problem: Robotic fruit pickers need to distinguish ripe (red) from unripe (green) fruit and determine the exact 3D shape for grasp planning without bruising. Current systems use fixed-threshold color analysis that fails under variable lighting. **Vision AI:** Instance segmentation with prompts "red tomato. green tomato. ripe strawberry. unripe strawberry." SAM 2 traces the exact curve of each fruit. **Agent Action:** The agent computes a "Ripeness Index" per row and generates a harvest schedule: "Row 4 is 90% ripe, send the picking crew tomorrow; Row 7 is 40% ripe, wait 3 days." **Dataset:** Tomato Ripeness Dataset (Kaggle) or MinneApple dataset (University of Minnesota). **Publishability:** Combining zero-shot segmentation with robotic graspability analysis is highly sought after in robotics journals (RA-L, CASE). The natural language control ("find ripe fruit") vs. fixed classifiers is the novelty hook.

11.3 Alternative 3: Sticky Trap Auditor (Pest Counting)

Problem: Farmers hang yellow sticky traps in greenhouses to monitor insect populations. Manual counting is tedious, error-prone, and infrequent, leading to delayed pest outbreak detection. **Vision AI:** Object detection and tiny-object segmentation with prompts "whitefly. aphid. thrips. fruit fly." Grounding DINO detects individual insects; SAM 2 segments each for size-based species differentiation. **Agent Action:** The agent counts insects per trap, compares against a "Pest Threshold Database" (stored in ChromaDB via SmartDesk's KnowledgeAgent RAG), and alerts: "Whitefly population exceeded safety limits in Greenhouse B. Recommended action: Apply neem oil within 48 hours." **Dataset:** IP102 (Insect Pest dataset, 102 classes, 75,000+ images) or D0 (Agricultural pest dataset). **Publishability:** Tiny object detection is notoriously difficult; proving the multi-agent refinement loop can accurately count tiny insects on noisy sticky trap backgrounds is a strong thesis contribution for CVPR workshops or ESA journals.

11.4 Alternative 4: Post-Harvest Grader (Conveyor Belt Sorting)

Problem: After harvest, produce (potatoes, apples, mangoes) must be sorted by quality. Bruised or rotten items contaminate the batch and reduce shelf life. Manual sorting is slow and inconsistent across shifts. **Vision AI:** Anomaly detection and segmentation with prompts "bruise. rot. fungal spot. healthy potato." Grounding DINO localizes defects; SAM 2 segments each defect precisely for severity grading. **Agent Action:** The agent acts as a virtual QA manager, generating a daily "Grade A vs Grade B" yield report, estimating shelf-life reduction per defect type, and triggering removal of Grade C items. **Dataset:** Kaggle Potato Defect Dataset or Apple Defect Dataset. **Publishability:** Post-harvest loss reduction directly addresses UN Sustainable Development Goal 12.3 (halve per-capita food waste by 2030). Papers with SDG impact receive favorable review at sustainability-focused venues (MDPI Sustainability, IFAC Agricontrol). The zero-shot angle: no retraining needed when switching from potato to mango inspection.

11.5 Alternative 5: Hydroponics Root Monitor (Indoor Farming)

Problem: In vertical farms and hydroponic systems, roots are submerged in opaque nutrient solution and cannot be visually inspected. Root rot (brown necrotic roots) develops silently and kills the crop before above-ground symptoms appear. **Vision AI:** Segmentation of water/root imagery with prompts "white healthy root. brown necrotic root. root rot." SAM 2 segments each root strand despite refraction and turbidity. **Agent Action:** The agent correlates root color

changes with mock IoT sensor data (pH, dissolved oxygen, EC) to predict disease 3 days before visible above-ground symptoms. When root rot is detected, it queries SmartDesk's KnowledgeAgent for treatment protocols from hydroponics manuals. **Dataset:** Likely requires a small custom dataset (50 photos of healthy and rotting roots), which is feasible for zero-shot: SAM 2 proves it works without training data on this novel domain. **Publishability:** "Controlled Environment Agriculture" (CEA) is booming globally with multi-billion dollar investment. Proving vision AI works underwater/in hydroponic conditions is novel and directly relevant to the CEA industry.

Alternative	Focus	Dataset	Publishability Hook
1: Drone Scout	Orthomosaic weed mapping	SugarBeets, WeedMap	Macro vs micro segmentation trend
2: Smart Harvester	Fruit ripeness grading	MinneApple, Tomato Kaggle	Robotics graspability + NL control
3: Sticky Trap	Insect pest counting	IP102, D0	Tiny object detection challenge
4: Post-Harvest	Conveyor belt sorting	Kaggle Potato/Apple Defect	UN SDG 12.3 food waste reduction
5: Hydroponics	Root rot monitoring	Custom (50 images)	CEA industry + underwater vision novelty

Table 15: Five alternative FYP ideas using the same technology stack.

12. 4-Phase Execution Plan & The "Secret Sauce"

This section provides a more granular execution plan organized into four phases, emphasizing what each team member delivers at each phase boundary. This structure ensures the project is always demonstrable at every phase review with Prof. Dhore, and that no single member becomes a bottleneck.

12.1 Phase 1: Foundation (Weeks 1-3)

Member 1 (Architect): Clone SmartDesk repo. Set up the GitHub repo with branches per member. Define the LangGraph GraphState extensions in state.py (add image, boxes, masks, spray_map fields). Set up CI with basic linting. **Member 2 (Vision):** Get Grounding DINO-T and SAM 2-Tiny running in a Google Colab notebook. Upload one picture of a weed, type "weed", and get a segmentation mask. **Milestone:** A working Colab notebook that segments one weed image end-to-end. **Member 3 (Data):** Download the DeepWeeds and PlantDoc datasets. Clean data (resize, verify labels). Write the first evaluation script: iou_dice.py that computes IoU and Dice between predicted and ground-truth masks. **Member 4 (UI):** Build a dummy Streamlit app with just a file uploader and a text box. No AI connected yet; focus on clean layout and responsive design. **Phase Review:** Show Prof. Dhore the Colab notebook segmenting a weed.

12.2 Phase 2: Core Magic (Weeks 4-7)

Member 2 (Vision): Wrap the Colab notebook into a Python API using FastAPI. The API accepts an image and text prompt, runs Grounding DINO + SAM 2, and returns the segmented image with masks as JSON. Deploy on Colab or a local server. **Member 1 (Architect):** Connect the Vision API to LangGraph. Implement the Orchestrator routing: if user query contains "segment" or "find," dispatch to the vision pipeline. If the vision agent detects "disease," trigger the RAG KnowledgeAgent for treatment lookup. **Member 3 (Data):** Feed 10 PDF farming manuals (from ICAR, University extension services) into ChromaDB via SmartDesk's KnowledgeAgent. Verify RAG retrieval quality: "How to treat tomato blight" should return relevant sections. **Member 4 (UI):** Connect the Streamlit frontend to the LangGraph backend via API calls. The app is now "alive": upload image, type prompt, see segmentation result. **Phase Review:** Live demo of the full pipeline on one weed image through the Streamlit UI.

12.3 Phase 3: Layman Features (Weeks 8-10)

Members 1 + 4: Build the "Inventory" and "Task" system. When the AI detects 5 weeds, it automatically deducts "50ml Herbicide" from a mock database and creates a "To-Do" task for the farmer. Implement the Report Agent that generates downloadable CSV and PDF spray maps. **Member 3:** Integrate a free Weather API (OpenWeatherMap) and agricultural commodity price API to populate the farmer dashboard with context. Run evaluation on SugarBeets and PlantDoc: record IoU, Dice, and the critical "Herbicide Saved %" metric. **Member 2:** Implement the **Refinement Loop** (the publishable hook). If SAM 2 predicted IoU is below 0.85, automatically add a corrective point at the mask error centroid and re-segment. Document this heavily with before/after examples for the paper. **Phase Review:** Live demo with spray map, inventory deduction, and refinement loop visualized.

12.4 Phase 4: Testing & Paper Writing (Weeks 11-16)

All Members: Run 100+ test images through the system across all 5 applications. Record accuracy (IoU, Dice, mAP), inference time, and the "Herbicide Saved %" for each application. Compute 95% confidence intervals and statistical significance for refinement ablation. **Writing assignments:** Member 1 writes the Architecture section (system diagram, LangGraph flow, GraphState extensions). Member 2 writes the Vision section (Grounding DINO + SAM 2 pipeline, refinement loop, ablation results). Member 3 writes the Literature Review and Evaluation sections (gap analysis, baseline comparisons, statistical tests, thesis figures). Member 4 writes the Application & Demo section (UI screenshots, user study if time permits, deployment notes).

12.5 The "Secret Sauce" for an A+

Do not just show the professor cool AI segmentation images. **Show the math.** At the end of the presentation, show a slide that says: *"By using our AgriAgent, a farmer reduces herbicide use by 73%."* Translating AI image segmentation into real-world money and environmental savings is exactly what gets FYPs published and graded highly. The key metrics to emphasize in every demo and paper section are: (1) Herbicide Savings % (target: 60-73%), (2) Zero-Shot Transfer Accuracy (same pipeline, 5 applications, no retraining), (3) Inference Speed (target: under 2.4 seconds end-to-end on T4), and (4) Refinement Improvement (Dice gain from the IoU-based loop with $p < 0.05$). The pitch to Prof. Dhere should be: *"Sir, we are building a Multi-Agent Precision Agriculture Framework. We use Grounding DINO and SAM 2 for Zero-Shot Image Segmentation for weed and disease detection. Unlike existing systems requiring weeks of retraining for new crops, our LangGraph-orchestrated agent allows a layman farmer to use natural language to instantly generate site-specific spray maps, update inventory, and retrieve agronomic advice, reducing chemical usage by up to 70%."*

13. 2026 Model Landscape & Defense

This section addresses the critical question that reviewers will raise: "Why use models from 2024 when it is 2026?" Both Grounding DINO and SAM 2 are already **vision-transformer models** (Swin Transformer and Hiera ViT, respectively), not legacy CNNs. However, newer alternatives have emerged that must be discussed to pre-empt reviewer objections.

13.1 SAM 3: Segment Anything with Concepts (Nov 2025)

SAM 3 (Meta AI, November 2025, 825 citations on OpenReview) is a unified foundation model that accepts **text prompts** (open-vocabulary noun phrases like "segment the weed") and image exemplar prompts, eliminating the need for bounding box inputs. SAM 3.1 (March 2026) adds faster real-time video segmentation. GitHub: <https://github.com/facebookresearch/sam3>. Ultralytics integration since October 2025. **Critical implication:** SAM 3 could theoretically collapse the two-stage pipeline into a single model. This is the strongest counter-argument a reviewer could raise. **However**, SAM 3 is significantly heavier than SAM 2-Tiny (38.9M params); the smallest SAM 3 variant requires ~4-6GB VRAM, leaving insufficient headroom for Grounding DINO + LangGraph on a free Colab T4 (16GB). For an FYP constrained to \$0 hardware, SAM 2-Tiny remains the pragmatic and defensible choice.

13.2 Grounding DINO Evolution

No formal "Grounding DINO 2" exists, but IDEA-Research released **Grounding DINO 1.5 Edge** (improved edge deployment) and **Dynamic-DINO** (ICCV 2025, MoE tuning for real-time open-vocabulary detection). The original Grounding DINO-T (172M params) remains the most documented variant with native HuggingFace Transformers support. **Recommended upgrade path:** Try Grounding DINO 1.5 Edge in Week 2; fall back to GDINO-T if compatibility issues arise on Colab T4.

13.3 VLM-Guided Refinement (2026 Novelty)

Qwen2.5-VL (January 2025, Alibaba, 3B/7B/72B params) processes images at native resolution with bounding box and point grounding. **Qwen3-VL-Seg** (May 2026) converts grounding priors into pixel-level segmentation. **Novel 2026 contribution:** Instead of a fixed heuristic (if IoU < 0.85), integrate a lightweight VLM (Qwen2.5-VL-3B via Groq API) as a "reasoning critic" in the refinement loop. The VLM examines the mask + image and decides whether segmentation is correct, suggesting corrections based on visual reasoning. This "VLM-guided refinement instead of RL" is directly comparable to MedSAM-Agent but simpler and more interpretable, providing a genuine 2026 novelty.

13.4 Strategic Options & Defense

Option	Risk	Pros	Defense
A: SAM 2 + GDINO-T	LOW	Runs on free T4, mature, bulletproof demo	Discuss SAM 3/VLMs in Limitations
B: GDINO 1.5 + SAM 2.1	LOW-MED	"Latest stable" stack, better zero-shot	Swap checkpoints; identical architecture
C: SAM 3 / unified VLM	HIGH	Cutting-edge but OOM risk on free T4	Only with paid A100 GPU access

Table 16: Strategic options for model selection.

Recommended: Option B (light upgrade, defended as Option A). Keep SAM 2-Tiny (T4-safe). Try GDINO 1.5 Edge; fall back if needed. Add VLM-guided refinement as 2026 novelty. Dedicate a paper subsection to "Why not SAM 3 / unified VLMs?" answering with compute constraints + precision trade-offs.

Defense for Prof. Dhore: "Both Grounding DINO and SAM 2 are vision-transformer foundation models, not legacy CNNs. We chose them because they run on free Colab T4 hardware, and our contribution is the agentic orchestration framework, not the underlying models. We discuss newer 2025-26 models like SAM 3 and GDINO 1.5 in our limitations section, and demonstrate VLM-guided refinement as a 2026 extension."